

ACTIVITY PERTEMUAN 4

NAMA : MAULANA ABDUL AZIZ

NPM : 50425612

KELAS : 1IA04

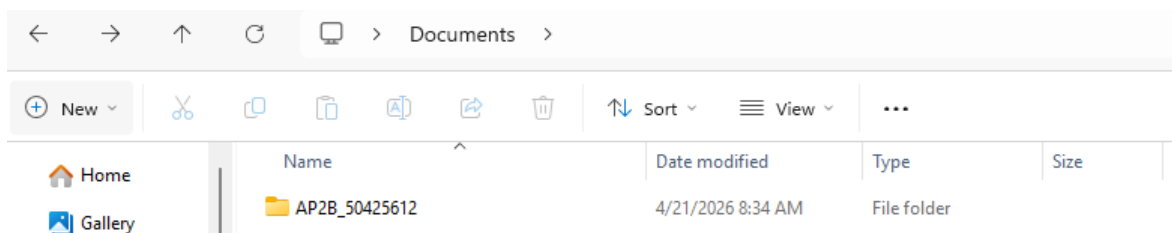
MATERI : FLASK

MATA PRAKTIKUM : ALGORITMA PEMROGRAMAN 2B

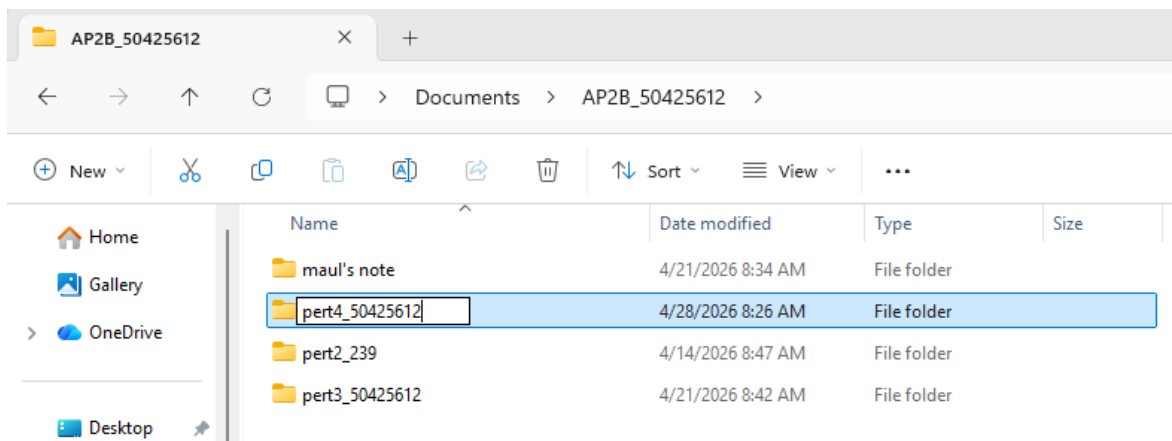
(Screenshoot langkah-langkah sesuai video pembelajaran dan jelaskan dengan ringkas)

1. Persiapan Struktur Folder

Masuk ke folder utama praktikum, yaitu **AP2B_50425612**.



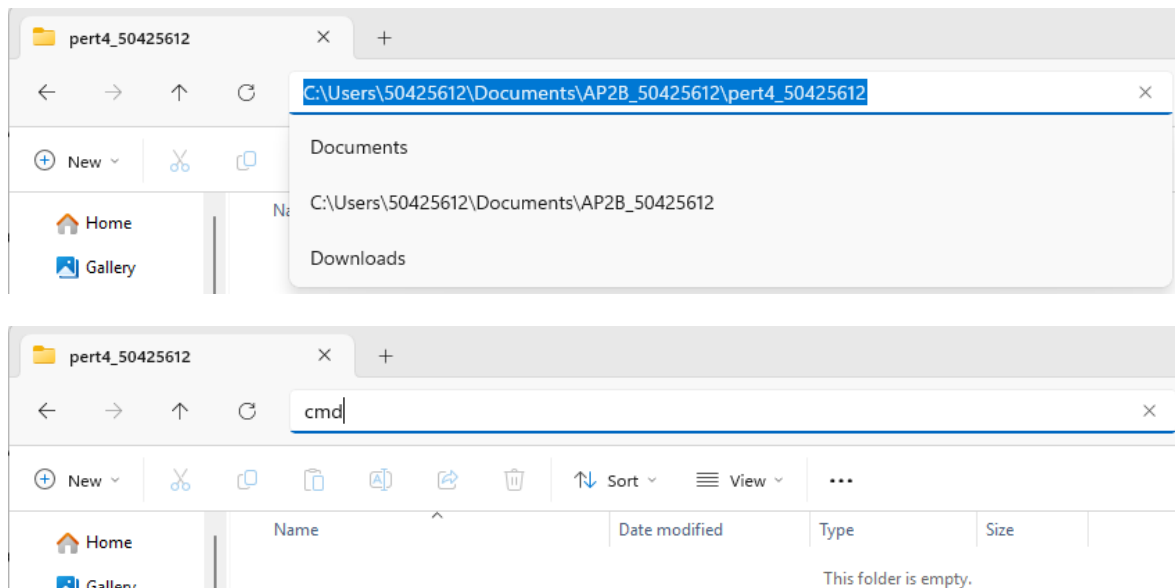
Membuat sub-folder baru dengan format penamaan **pert4_50425612** sebagai wadah file proyek pertemuan ini.



Logikanya, pemisahan folder ini dilakukan agar manajemen file antar pertemuan tetap rapi dan terstruktur.

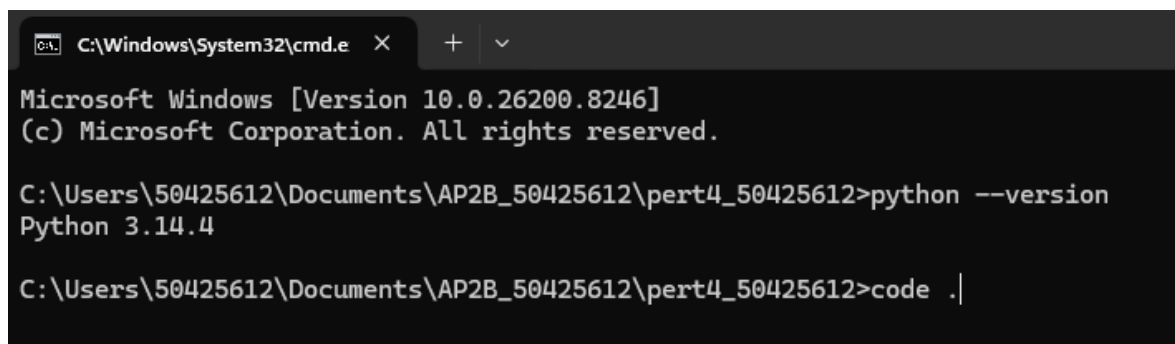
2. Membuka VS Code

Klik pada *address bar* folder yang telah dibuat, ketik **cmd**, lalu tekan Enter untuk membuka Command Prompt pada direktori tersebut.



Di dalam terminal, ketik perintah `python --version` untuk memastikan Python (minimal versi 3.11) sudah terpasang dengan benar. Dalam dokumentasi ini, versi yang terdeteksi adalah **3.14.4**.

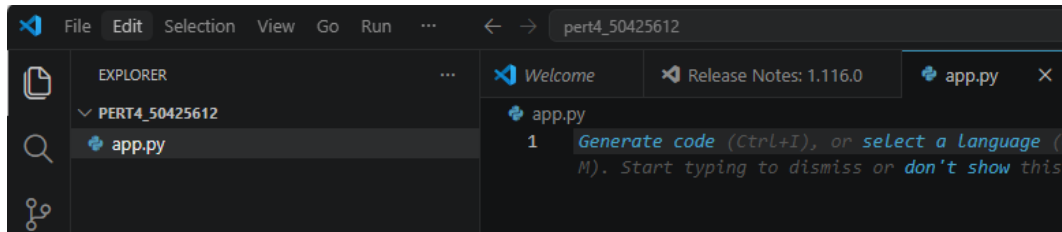
Ketik perintah `code .` untuk membuka folder tersebut secara otomatis ke dalam **Visual Studio Code**.



3. Setup file dan buat program

Di dalam VS Code, buat file Python baru dengan nama **app.py**.

Logikanya, file ini akan berfungsi sebagai server utama yang menangani logika API dan pengelolaan data buku secara terpusat.



Tuliskan kode program dasar dengan mengimpor Flask, jsonify, request, dan json, lalu inisialisasi aplikasi Flask.

Deklarasikan variabel datas sebagai *list of dictionaries* yang berisi katalog buku awal (contoh: *Madilog* dan *Animal Farm*) mencakup ID, judul, penulis, dan harga .

Buat fungsi helper `format_rupiah(amount)` untuk mengubah angka harga menjadi format mata uang Rupiah agar lebih mudah dibaca oleh pengguna.

```
app.py
app.py > format_rupiah
1  from flask import Flask, jsonify, request
2  import json
3  app = Flask(__name__)
4  datas = [
5      {
6          'id': 1,
7          'items': [
8              { 'title': 'Madilog',
9                'author': 'Tan Malaka',
10               'price': 84000
11             }
12          ]
13      }, {
14          'id': 2,
15          'items': [
16              {
17                  'title': 'Animal Farm',
18                  'author': 'George Orwell',
19                  'price': 50000
20              }
21          ]
22      }
23  ]
24  def format_rupiah(amount):
25      return f"Rp{amount:,.0f}".replace(",", ".")
```

4. Implementasi Route CRUD (Create, Read, Delete)

Menyusun fungsi-fungsi untuk mengelola operasional data melalui protokol HTTP:

Method GET:

- Membuat rute /book (fungsi getAll) untuk menampilkan seluruh daftar buku yang tersedia.
- Membuat rute /book/<int:id> (fungsi getById) untuk mencari dan menampilkan buku spesifik berdasarkan parameter ID yang dimasukkan.

Method POST: Menambahkan fungsi addBook() pada rute /book untuk menerima data JSON baru dari pengguna, lengkap dengan validasi tipe data agar input tetap konsisten dan aman.

Method DELETE: Mengimplementasikan rute /book/<int:id> dengan fungsi deleteBook(id) untuk menghapus entri data dari server berdasarkan ID yang dipilih.

Logikanya, setiap rute menggunakan jsonify agar respon dikirim dalam format JSON yang standar dan universal.

```

27 @app.route('/book')
28 def getAll():
29     formatted_data = []
30     for data in datas:
31         formatted_data = {
32             'id': data['id'],
33             'items': []
34         }
35         for item in data['items']:
36             formatted_item = {
37                 'title': item['title'],
38                 'author': item['author'],
39                 'price': format_rupiah(item['price'])
40             }
41             formatted_data['items'].append(formatted_item)
42         formatted_data.append(formatted_data)
43     return jsonify(formatted_data)
44 @app.route('/book/<int:id>')
45 def getById(id):
46     for data in datas:
47         if data['id'] == id:
48             formatted_data = {
49                 'id': data['id'],
50                 'items': []
51             }
52             for item in data['items']:
53                 formatted_item = {
54                     'title': item['title'],
55                     'author': item['author'],
56                     'price': format_rupiah(item['price']) # Format harga disini
57                 }
58                 formatted_data['items'].append(formatted_item)
59             return jsonify(formatted_data)
60     return jsonify({'message': 'Data not found'})
61 @app.route('/book', methods=['POST'])
62 def addBook():
63     req_data = request.get_json()
64
65     if not all(key in req_data for key in ['id', 'items']):
66         return jsonify({'message': 'Missing required fields (id, items)'}), 400
67
68     if not isinstance(req_data['items'], list):
69         return jsonify({'message': '"items" should be a list'}), 400
70     for item in req_data['items']:
71         if not all(k in item for k in ['title', 'author', 'price']):
72             return jsonify({'message': 'Each item in \'items\' should contain title, author, and price'}), 400
73         if not isinstance(item['price'], (int, float)):
74             return jsonify({'message': 'Price must be a number'}), 400
75
76     new_data = {
77         'id': req_data['id'],
78         'items': req_data['items']
79     }
80     datas.append(new_data)
81     return jsonify(new_data), 201
82 @app.route('/book/<int:id>', methods=['DELETE'])
83 def deleteBook(id):
84     global datas
85     for index, data in enumerate(datas):
86         if data['id'] == id:
87             del datas[index]
88             return jsonify({'message': 'Book deleted successfully'}), 200
89     return jsonify({'message': 'Book not found'}), 404

```

Tambahkan rute dasar (/) yang akan memberikan respon sapaan "Halo Maulana Abdul Aziz !" saat server diakses.

```

90
91 @app.route('/')
92 def home():
93     return "Halo Maulana Abdul Aziz !"
94
95 if __name__ == '__main__':
96     app.run(debug=True)

```

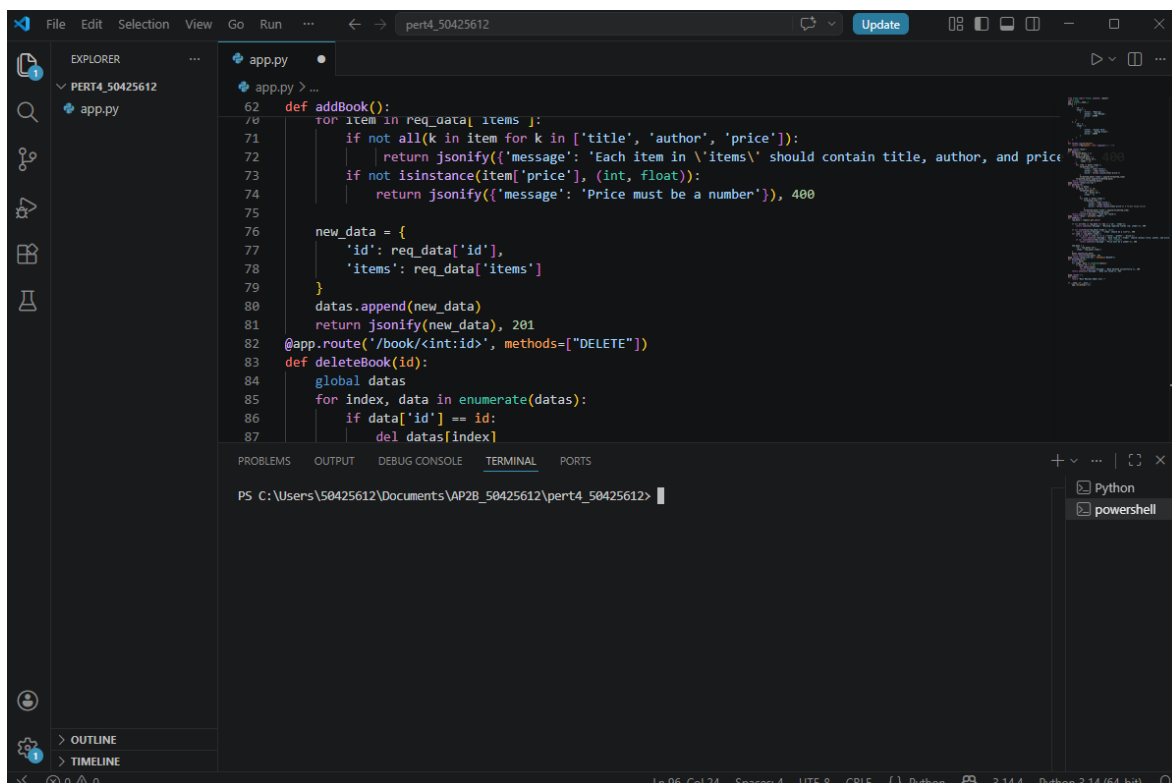
Jalankan server Flask dengan perintah `app.run(debug=True)` di bagian akhir kode.

Logikanya, mode *debug* diaktifkan agar server otomatis melakukan *restart* setiap kali ada perubahan pada kode, memudahkan proses pengembangan.

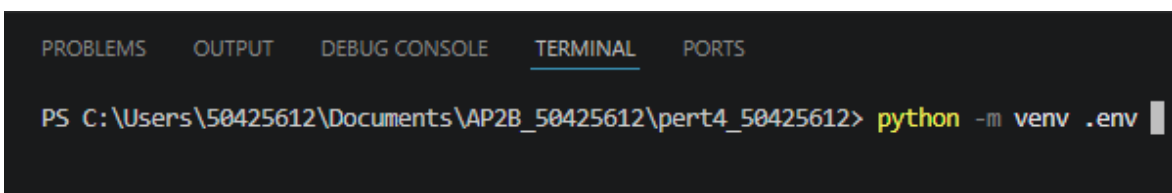
5. Konfigurasi Lingkungan Virtual (Venv) dan Instalasi

Mempersiapkan isolasi lingkungan agar dependensi proyek tidak tercampur:

Buka terminal internal VS Code (Ctrl + Shift + ') dan ketik perintah `python -m venv .env` untuk membuat *virtual environment* baru.

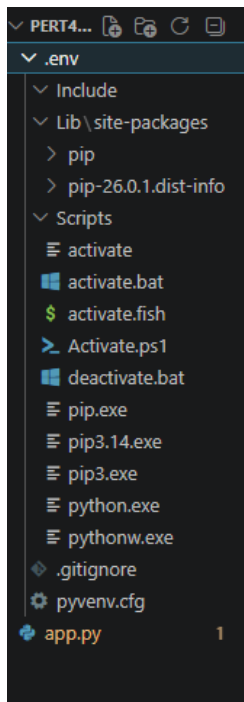


The screenshot shows the VS Code interface. The Explorer pane on the left shows a project named 'PERT4_50425612' with a file 'app.py'. The main editor area displays the code for 'app.py', which includes a Flask application with an 'addBook' endpoint and a 'deleteBook' endpoint. The terminal pane at the bottom shows the command prompt 'PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612>'.

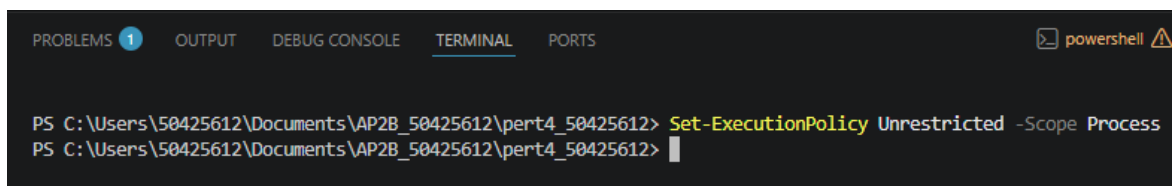


This is a close-up of the terminal window from the previous screenshot. It shows the command `python -m venv .env` being entered at the prompt 'PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612>'.

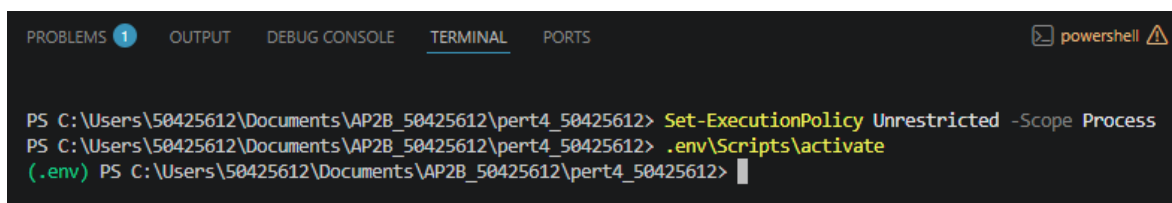
Atur kebijakan eksekusi terminal dengan perintah `Set-ExecutionPolicy Unrestricted -Scope Process` agar skrip aktivasi dapat dijalankan.



Aktivasi lingkungan dengan mengetik `.env\Scripts\activate`. Tanda `(.env)` pada terminal menunjukkan lingkungan sudah aktif.



Mengaktivasi dengan `activate`, `.env` adalah tanda program kita sudah berada dalam environmentnya



Jalankan perintah `pip install Flask` untuk memasang framework Flask ke dalam lingkungan virtual tersebut.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell

PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612> .env\Scripts\activate
(.env) PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612> pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Downloading click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->Flask)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.3 colorama-0.4.6 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 26.0.1 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.env) PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612>

```

6. Aktivasi Server dan Pengujian API

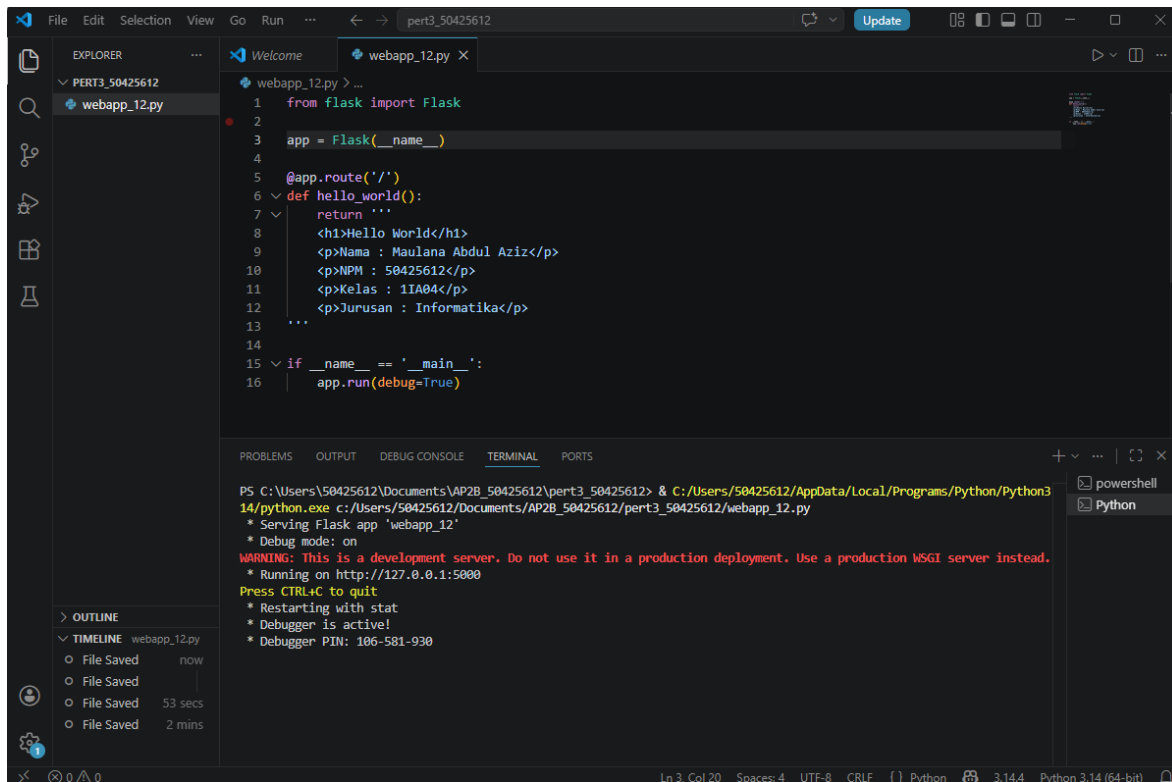
Tahap akhir untuk menjalankan dan memverifikasi aplikasi:

- Jalankan server melalui terminal dengan perintah `python app.py`.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python ⚠ + - □
```

```
(.env) PS C:\Users\50425612\Documents\AP2B_50425612\pert4_50425612> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 493-029-769
```

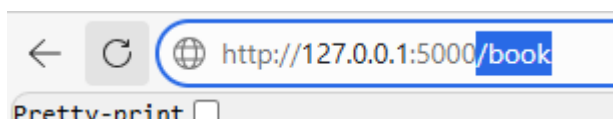
- Setelah muncul pesan Running on `http://127.0.0.1:5000`, buka tautan tersebut di browser untuk melihat hasilnya.

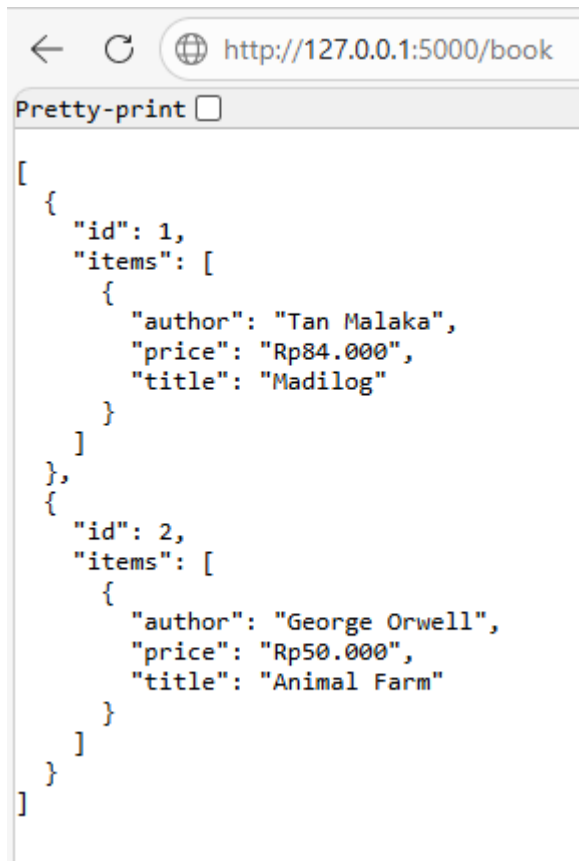


- Tambahkan rute dasar (/) yang memberikan respon sapaan "Halo Maulana Abdul Aziz!" sebagai indikator server berjalan.



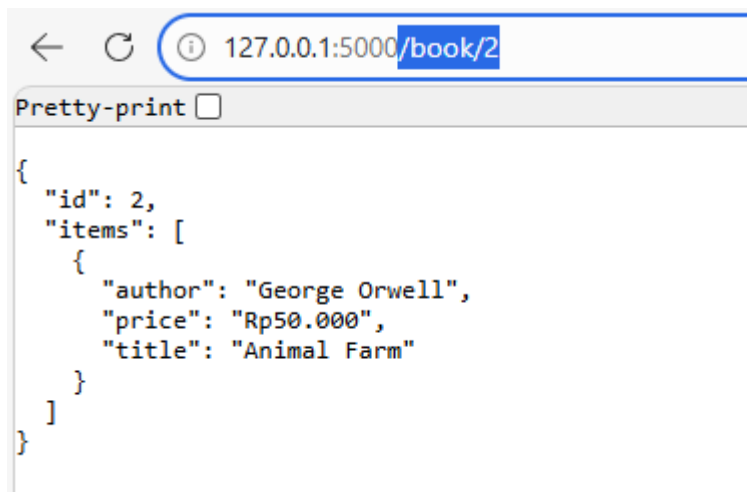
- Lakukan pengujian rute:
 - Akses <http://127.0.0.1:5000/book> untuk melihat seluruh data dalam format JSON yang sudah rapi (*pretty-print*).





```
[
  {
    "id": 1,
    "items": [
      {
        "author": "Tan Malaka",
        "price": "Rp84.000",
        "title": "Madilog"
      }
    ]
  },
  {
    "id": 2,
    "items": [
      {
        "author": "George Orwell",
        "price": "Rp50.000",
        "title": "Animal Farm"
      }
    ]
  }
]
```

- Akses <http://127.0.0.1:5000/book/2> untuk memverifikasi pencarian data berdasarkan ID.



```
{
  "id": 2,
  "items": [
    {
      "author": "George Orwell",
      "price": "Rp50.000",
      "title": "Animal Farm"
    }
  ]
}
```

- Logikanya, penggunaan mode debug=True memungkinkan server melakukan *restart* otomatis setiap ada perubahan kode, sangat memudahkan proses pengembangan.